

AUTOMATED CUSTOMER REVIEW SENTIMENT ANALYSIS SYSTEM

Design, Development, Automation and Evaluation

Formal Internship Project Report

Submitted to Learning Saint

Project title	Sentiment Analysis Project
Development environment	Python 3.13, VS Code and macOS
Submission date	31 July 2026

Learning Saint Internship Programme

Nairobi, Kenya

EXECUTIVE SUMMARY

This report documents the design, implementation and evaluation of an automated customer review sentiment analysis system developed as an internship project for Learning Saint. The client requirement was to classify incoming feedback as positive, neutral or negative, display results through an intuitive interface, process uploaded files automatically, notify the business when negative feedback is high, save reusable model artifacts and support periodic retraining [1]. The resulting solution was developed in Python using pandas, scikit-learn, NLTK, Matplotlib, Tkinter, Pillow, joblib and pytest within Visual Studio Code on macOS.

The Twitter US Airline Sentiment dataset was sourced from Kaggle because it contains customer-service feedback with the three required sentiment labels [2]. The source contained 14,640 records. After selecting the review and label fields, removing missing and duplicate entries, and applying text cleaning, 14,069 usable reviews remained. The final distribution was imbalanced: 9,043 negative reviews (64.28%), 2,880 neutral reviews (20.47%) and 2,146 positive reviews (15.25%). This imbalance informed the evaluation strategy: accuracy was reported, but macro precision, macro recall, macro F1-score, weighted F1-score, classification reports and confusion matrices were also generated so that minority-class performance was not hidden by the dominant negative class.

The preprocessing pipeline decoded HTML entities, removed tags, URLs and account mentions, retained hashtag words, normalised case and contractions, tokenised the text, removed English stopwords while preserving negation terms, and lemmatised tokens using WordNet. Two feature representations - Bag of Words and TF-IDF - were evaluated with Multinomial Naive Bayes and Linear Support Vector Classification. A stratified 80:20 split produced 11,255 training reviews and 2,814 testing reviews. Four configurations were compared using the same test set.

TF-IDF with Linear SVM achieved the strongest overall performance: 78.50% accuracy, 71.37% macro precision, 71.27% macro recall, 71.28% macro F1-score and 78.34% weighted F1-score. It exceeded the approximate 64.28% majority-class baseline and produced the highest macro F1-score among the tested configurations. The selected model and fitted vectorizer were persisted as `sentiment_model.pkl` and `vectorizer.pkl` for reuse by the graphical interface and automation modules.

The final application provides manual review prediction, CSV/JSON upload, sentiment summary cards, a review table, a distribution chart, result export, configurable negative-review alerts, automated incoming-folder monitoring and controlled model retraining. The automation successfully detected a new feedback file, classified its reviews, saved the results and archived the source. The completed test suite contained 18 automated tests, all of which passed. The system meets the assignment's functional requirements and provides a sound local prototype; however, performance may decline outside airline-related English feedback, and production deployment would require stronger data storage, security, messaging and monitoring architecture.

Key outcome: A complete end-to-end sentiment analysis prototype was delivered, with the best model achieving 78.50% test accuracy and a 71.28% macro F1-score, while all 18 automated software tests passed.

CONTENTS

- 1. Introduction**
- 2. Project Requirements and Scope**
- 3. Dataset and Exploratory Analysis**
- 4. Methodology**
- 5. Results and Evaluation**
- 6. System Implementation and Screenshots**
- 7. Limitations and Future Improvements**
- 8. Conclusion and Recommendations**

References

Appendices

List of Figures

- Figure 1. End-to-end sentiment analysis architecture
- Figure 2. Final cleaned dataset sentiment distribution
- Figure 3. Average review length by sentiment
- Figure 4. Model performance comparison
- Figure 5. Model-training terminal evidence
- Figure 6. Actual Tkinter dashboard after batch file processing
- Figure 7. Automated incoming-file processing evidence
- Figure 8. Controlled model retraining workflow
- Figure 9. Automated test-suite evidence

List of Tables

- Table 1. Project requirements and implemented components
- Table 2. Software tools and versions
- Table 3. Dataset preparation summary
- Table 4. Final sentiment distribution
- Table 5. Model training and testing split
- Table 6. Model comparison results
- Table 7. Selected model metrics
- Table 8. Functional verification summary

1. INTRODUCTION

1.1 Background

Sentiment analysis is a Natural Language Processing task that identifies the attitude or polarity expressed in text. In customer-service settings, the practical output is commonly represented as positive, neutral or negative. The technique allows businesses to convert large volumes of unstructured feedback into measurable information that can support service improvement, complaint prioritisation and management reporting.

Manual review of customer comments is difficult to sustain when feedback arrives through several channels. Human reading is valuable for nuanced cases, but it is slow, costly and potentially inconsistent when applied to every incoming message. An automated classifier can provide a first-level categorisation, highlight unusual increases in negative feedback and allow staff to focus their attention on the most important records. The Learning Saint brief therefore required not only model training, but also an operational interface, incoming-data automation, alerts, saved artifacts and retraining support [1].

1.2 Problem Statement

Businesses need a repeatable method for understanding customer sentiment without manually reviewing every item. The problem addressed in this project was to build a local automated system that accepts individual reviews and review files, cleans the text consistently, classifies each review into one of three sentiment classes, summarises the resulting distribution and makes the output available for action. The solution also needed to cope with new incoming files and provide a controlled method for adapting the model as newly labelled feedback becomes available.

1.3 Project Aim

The aim was to design and implement an automated customer review sentiment analysis system that classifies English feedback as positive, neutral or negative and presents the results through a user-friendly desktop dashboard.

1.4 Specific Objectives

1. Obtain and prepare a labelled customer-feedback dataset with positive, neutral and negative classes.
2. Develop a reusable preprocessing pipeline for cleaning, tokenising and lemmatising review text.
3. Compare Bag of Words and TF-IDF feature extraction with Multinomial Naive Bayes and Linear SVM classifiers.
4. Evaluate model performance using accuracy, precision, recall, F1-score, classification reports and confusion matrices.
5. Save the selected model and vectorizer for later inference.
6. Build a Tkinter interface for individual and batch predictions, statistics, visualisation and export.
7. Automate the processing of incoming CSV and JSON files and trigger alerts when negative sentiment is high.
8. Implement controlled model retraining and an automated software test suite.

1.5 Report Structure

The report first maps the assignment requirements to the implemented components, then describes the dataset, exploratory findings and methodology. It presents the comparative model results, documents the user interface and automation evidence, evaluates limitations and closes with conclusions and recommendations. Supporting commands and the project structure are provided in the appendices.

2. PROJECT REQUIREMENTS AND SCOPE

2.1 Assignment Requirements

The supplied project brief defined a broader automation system rather than a model-only exercise. It required labelled feedback in CSV or JSON format, a preprocessing pipeline, a suitable sentiment classifier, automated handling of new reviews, an interface displaying review text and sentiment distribution, negative-review notifications, saved model files and documentation explaining setup, deployment and retraining [1].

Requirement	Implemented approach	Status
Labelled dataset	Kaggle airline feedback was standardised to review and sentiment fields.	Completed
Text preprocessing	HTML, URLs, handles, case, tokenisation, stopwords, negation and lemmatisation.	Completed
Feature extraction	Bag of Words and TF-IDF with unigram and bigram features.	Completed
Model comparison	Multinomial Naive Bayes and Linear SVM were evaluated.	Completed
Saved artifacts	sentiment_model.pkl, vectorizer.pkl and model metadata.	Completed
Dashboard	Manual prediction, batch upload, statistics, table, chart and export.	Completed
Automation	Incoming-folder monitoring, processing, saving and archiving.	Completed
Alerts	Local dashboard warning and alert log based on a negative threshold.	Completed
Retraining	Candidate evaluation, model backup and conditional promotion.	Completed
Testing	18 automated tests covering core modules.	Completed

Table 1. Project requirements and implemented components.

2.2 Scope

The implemented solution is a local desktop prototype. It processes English-language review text through manual entry, CSV upload, JSON upload or monitored folders. It produces one overall sentiment label for each review. The scope includes model persistence, batch prediction, export, alerts, file archiving, model retraining

and automated tests. It does not include live integration with a commercial review platform, a production database, multi-user authentication, cloud deployment or external email/SMS delivery.

2.3 Development Environment

Component	Version or platform
Operating system	macOS
Editor	Visual Studio Code
Python	3.13.5
pandas	3.0.5
NumPy	2.5.1
scikit-learn	1.9.0
NLTK	3.10.0
Matplotlib	3.11.1
Tkinter	8.6
Testing	pytest 9.1.1

Table 2. Software tools and versions confirmed in the project environment.

3. DATASET AND EXPLORATORY ANALYSIS

3.1 Dataset Selection

The Twitter US Airline Sentiment dataset was selected from Kaggle because it contains authentic customer-service comments and an existing three-class label: negative, neutral or positive [2]. Although the comments originated from Twitter, they represent customer feedback concerning airline products and services and therefore satisfy the assignment's need for labelled client reviews. The source file was retained separately from the processed data so that data preparation remained reproducible.

3.2 Data Preparation

The original file contained 14,640 observations. Only the text and airline_sentiment fields were retained and renamed review and sentiment. Whitespace and labels were standardised, missing values and empty reviews were removed, labels were validated, and duplicate review text was excluded. This produced 14,427 prepared records. After full text preprocessing, empty cleaned reviews and duplicate cleaned text-label combinations were removed, leaving 14,069 records.

Dataset stage	Records
Original source records	14,640
Prepared records after field selection and validation	14,427
Records removed during initial preparation	213
Final cleaned records	14,069
Additional records removed during text cleaning	358

Table 3. Dataset preparation summary.

3.3 Class Distribution

Sentiment	Reviews	Share
Negative	9,043	64.28%
Neutral	2,880	20.47%
Positive	2,146	15.25%
Total	14,069	100.00%

Table 4. Final sentiment distribution.

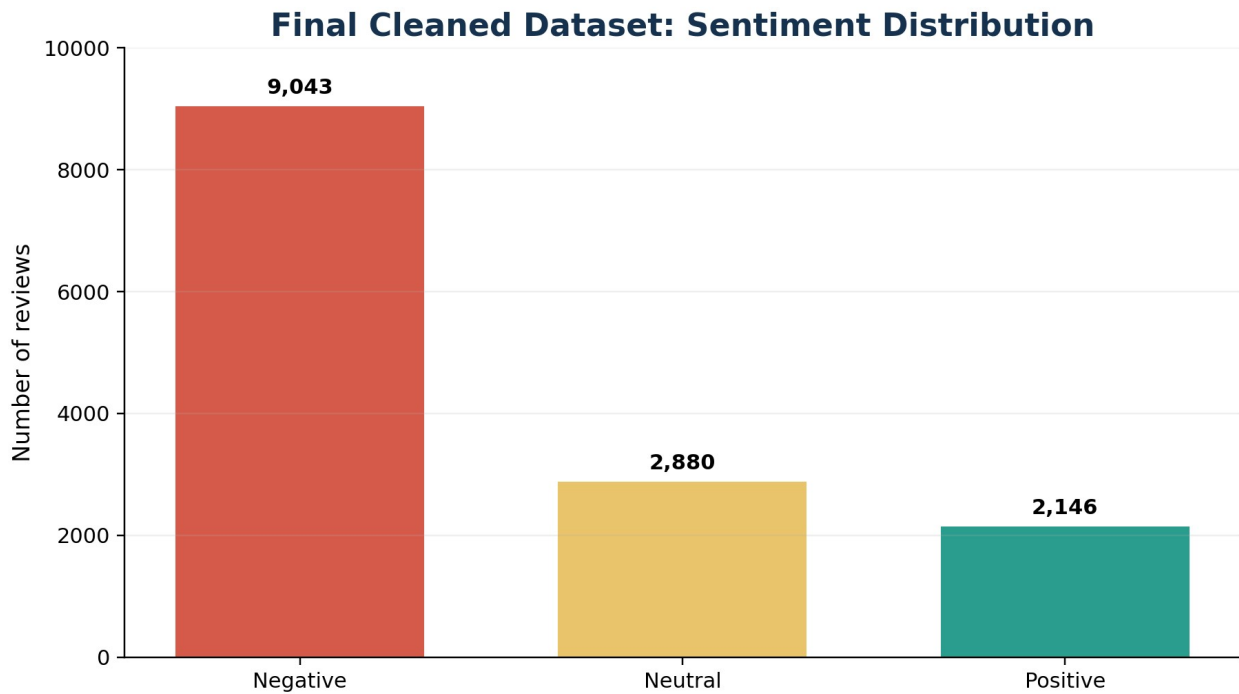


Figure 2. Distribution of the 14,069 reviews retained after cleaning.

The distribution shows a strong negative-class majority. A classifier predicting every review as negative would achieve approximately 64.28% accuracy, even though it would provide no useful detection of neutral or positive feedback. Consequently, macro F1-score was used as the primary model-selection measure because it gives equal weight to each sentiment class. Weighted F1-score and accuracy were retained as complementary measures.

3.4 Review Length

The average original review contained 17.99 words and 105.58 characters. After preprocessing, the average fell to 9.12 retained tokens. Negative reviews were longer on average than neutral and positive reviews: 19.74 words compared with 14.79 and 14.92 words respectively. This may reflect the tendency for dissatisfied customers to explain delays, cancellations, lost luggage or service failures in greater detail.

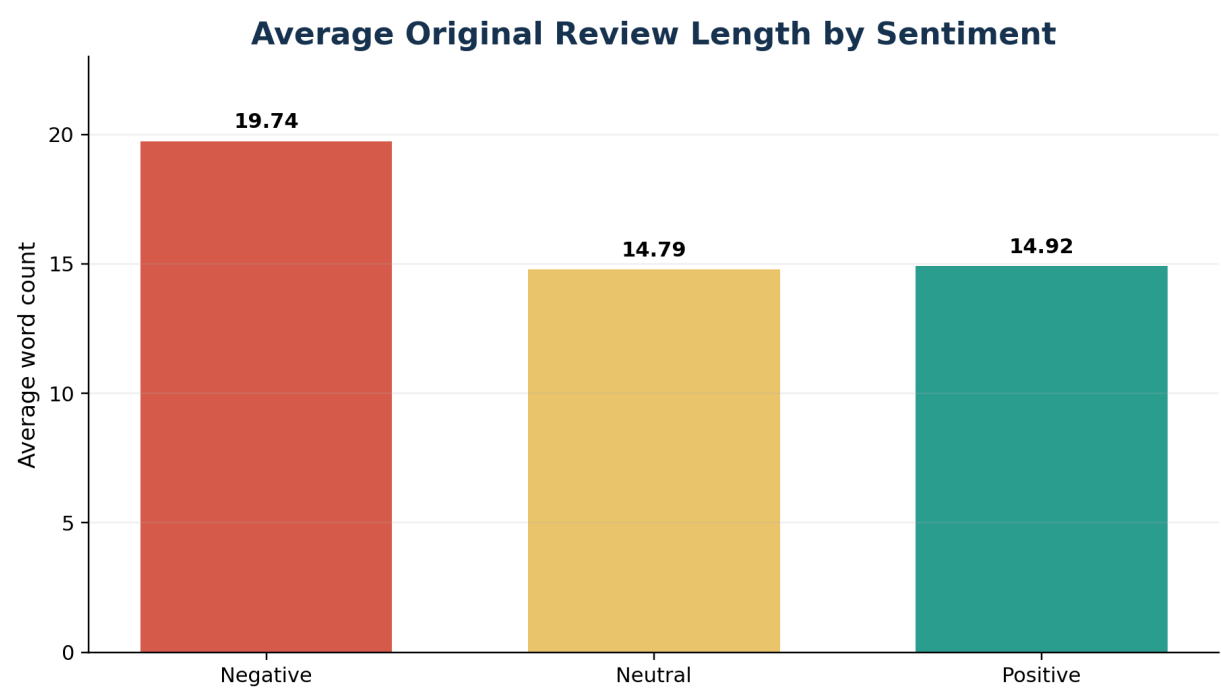


Figure 3. Average original word count by sentiment class.

4. METHODOLOGY

4.1 System Architecture

The system was designed as a modular pipeline. Review text enters through the Tkinter dashboard, an uploaded CSV/JSON file or the incoming-folder monitor. The same preprocessing function is applied in every route. The saved TF-IDF vectorizer transforms cleaned text to a sparse numeric matrix, and the saved Linear SVM assigns the sentiment. Results are returned to the dashboard or written to a processed output file. Negative-review statistics are then assessed against the configured threshold.

End-to-End Sentiment Analysis Architecture



The same saved preprocessing, vectorizer and model artifacts are reused for manual and batch predictions.

Figure 1. End-to-end sentiment analysis architecture.

4.2 Text Preprocessing

Consistent preprocessing was essential because the model was trained on short social-media feedback containing account names, links, HTML entities, punctuation and informal contractions. The implemented process performed the following operations:

- Decoded HTML entities such as & into readable characters.
- Removed HTML tags and web links.
- Removed Twitter account mentions while retaining meaningful hashtag text.
- Converted text to lowercase and normalised whitespace.
- Expanded common negative contractions so that terms such as did not were preserved.
- Retained letters, apostrophes and spaces while removing unnecessary symbols.
- Tokenised text with NLTK TweetTokenizer.
- Removed English stopwords but preserved no, not, never and related negation terms.
- Lemmatised tokens using WordNet, primarily with the verb part of speech.
- Removed cleaned reviews that became empty or duplicated existing records.

NLTK's WordNetLemmatizer applies WordNet's morphological function to return a base form when available [7]. The decision to preserve negation was important: removing not from not good would incorrectly leave good and reverse the intended meaning.

```
Original: @VirginAmerica I didn't enjoy the delayed flight at all!
Cleaned: not enjoy delay flight
```

4.3 Feature Extraction

Machine-learning classifiers require numeric input. Two sparse feature representations were evaluated. CountVectorizer produced Bag-of-Words counts, while TfidfVectorizer converted documents to TF-IDF features

[4]. Both configurations used unigrams and bigrams, a maximum of 20,000 features, a minimum document frequency of 2 and a maximum document frequency of 0.95. The TF-IDF configuration additionally used sublinear term frequency.

Unigrams capture individual words such as delay or excellent, while bigrams capture short phrases such as not helpful or great service. Restricting the vocabulary reduced noise and computational cost. The fitted vocabulary contained 13,527 features for both representations after the training split was applied.

4.4 Classifiers

Two traditional classifiers were compared. Multinomial Naive Bayes is a classic text-classification method designed for multinomially distributed features and is commonly applied to word counts; scikit-learn also notes that TF-IDF inputs can work in practice [5]. The model was configured with $\alpha = 0.5$.

LinearSVC performs linear support vector classification, supports sparse input and handles multiclass classification using a one-versus-rest strategy [6]. It was configured with balanced class weights to reduce the influence of the majority negative class, `random_state = 42` and `max_iter = 5,000`. Linear SVM was suitable for the high-dimensional sparse feature space produced by text vectorisation.

4.5 Train-Test Split and Evaluation

The final dataset was split into 80% training and 20% testing subsets using stratification. The test set was not used to fit the vectorizer or classifier. The fixed random state made the comparison reproducible.

Subset	Records	Class counts
Training	11,255	Negative 7,234; Neutral 2,304; Positive 1,717
Testing	2,814	Negative 1,809; Neutral 576; Positive 429

Table 5. Stratified model training and testing split.

The evaluation metrics were accuracy, macro precision, macro recall, macro F1-score, weighted precision, weighted recall and weighted F1-score. A classification report was generated for each configuration; scikit-learn's `classification_report` provides precision, recall, F1-score and support by class together with macro and weighted averages [9]. Confusion matrices were also saved to show how predictions were distributed across the three classes.

4.6 Model Persistence

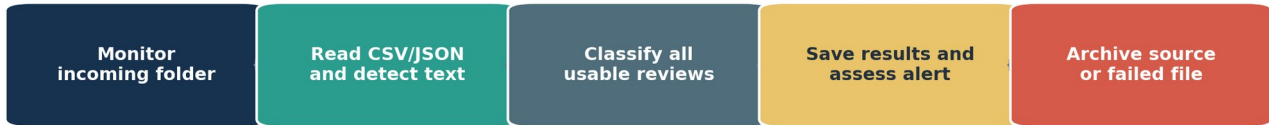
The best model and its fitted vectorizer were saved with joblib as `models/sentiment_model.pkl` and `models/vectorizer.pkl`. Model metadata, including the selected configuration, metrics, data split and timestamp, was saved in JSON format. Persisting both objects was necessary because prediction must use the same learned vocabulary and inverse-document-frequency values used during training.

4.7 Interface, Automation and Alerts

The desktop interface was implemented with Tkinter, Python's standard interface to the Tcl/Tk GUI toolkit [8]. It accepts a single review, predicts its sentiment, appends the result to a table, updates summary cards and redraws a sentiment chart. CSV and JSON uploads are supported, and common column names such as review, text, feedback and comment are detected automatically. Results can be exported as CSV or JSON.

The automation module monitors data/incoming for supported files. New files are read, their review column is detected, predictions are generated, results are saved, the negative-review share is assessed and the source file is archived. A warning is triggered when at least five reviews are processed and at least 50% are negative. Invalid files are moved to a failed archive.

Automated Incoming-File Workflow



The folder watcher checks at a configurable interval and processes new files without manual intervention.

Automated incoming-file workflow used by the local folder monitor.

4.8 Controlled Retraining

A retraining module was added so that new labelled feedback can improve the classifier without automatically replacing a stronger production model. The module validates labels, cleans the reviews, excludes overlaps with the protected original test set, trains a candidate using the existing model family, and compares the current and candidate macro F1-scores on the same test set. If the candidate meets the configured threshold, the existing artifacts are backed up and the new model is deployed; otherwise the current model is retained.

Controlled Model Retraining Workflow



A candidate model is promoted only when it meets the configured performance requirement.

Figure 8. Controlled retraining and model-promotion workflow.

5. RESULTS AND EVALUATION

5.1 Comparative Model Results

Features	Model	Accuracy	Macro precision	Macro recall	Macro F1	Weighted F1
TF-IDF	Linear SVM	0.7850	0.7137	0.7127	0.7128	0.7834
Bag of Words	Linear SVM	0.7715	0.6985	0.7118	0.7047	0.7740
Bag of Words	Multinomial NB	0.7761	0.7139	0.6747	0.6909	0.7675
TF-IDF	Multinomial NB	0.7381	0.7695	0.5280	0.5679	0.6903

Table 6. Model comparison results on the held-out test set.

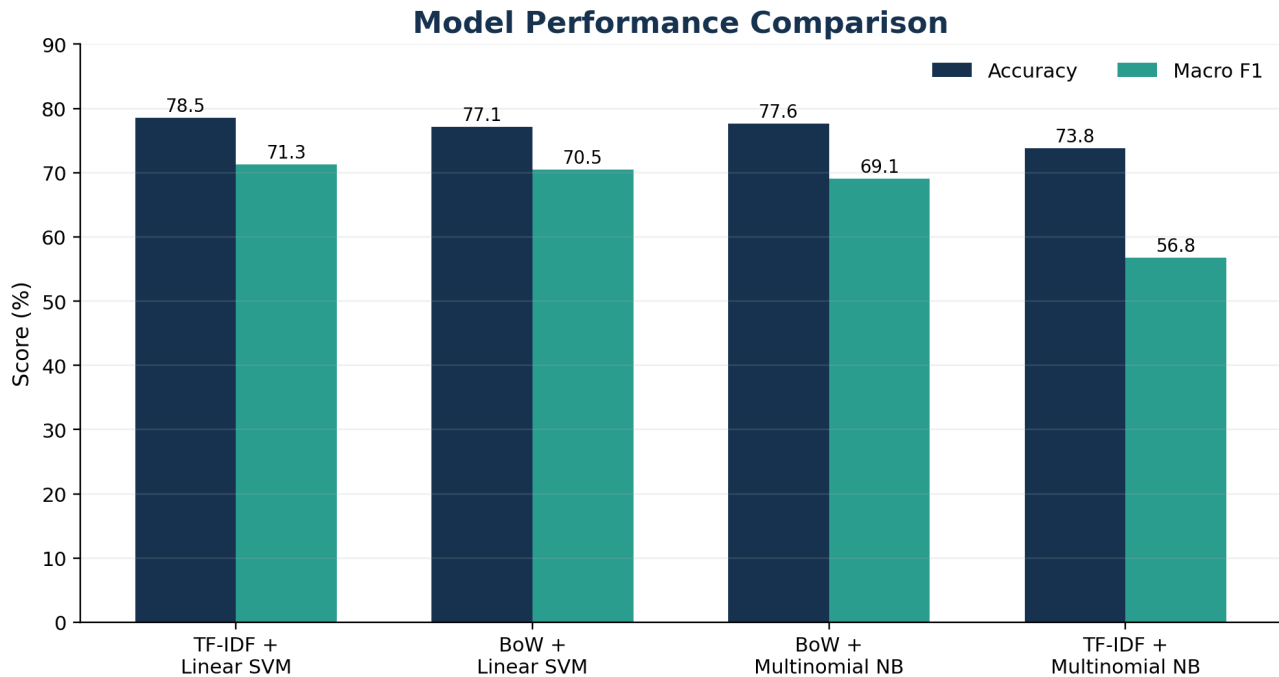


Figure 4. Accuracy and macro F1-score for the four model configurations.

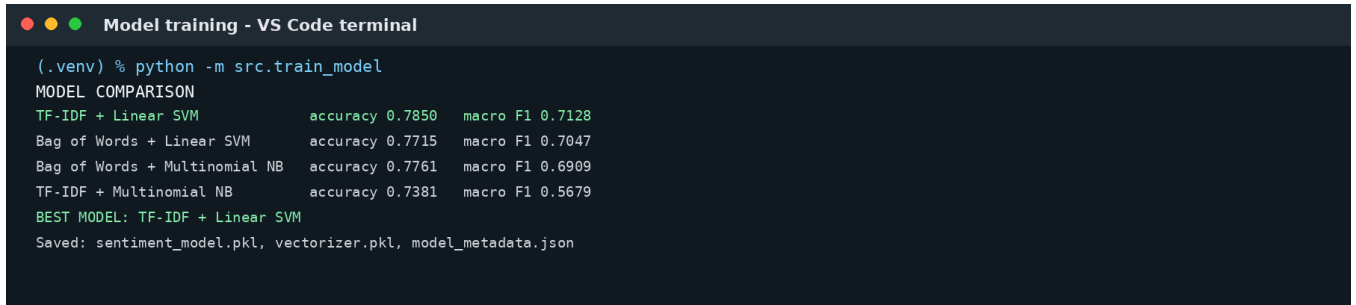
TF-IDF with Linear SVM achieved both the highest accuracy and the highest macro F1-score. Bag of Words with Linear SVM was second on macro F1 and produced a similar macro recall. Bag of Words with Multinomial Naive Bayes achieved slightly higher accuracy than the Bag-of-Words SVM but lower macro recall and macro F1. TF-IDF with Multinomial Naive Bayes had the highest macro precision but the weakest macro recall, indicating conservative predictions that missed a substantial share of minority-class observations.

5.2 Selected Model

Metric	Result
Accuracy	78.50%
Macro precision	71.37%
Macro recall	71.27%
Macro F1-score	71.28%
Weighted F1-score	78.34%
Vocabulary size	13,527 features

Table 7. Performance of the selected TF-IDF and Linear SVM model.

The improvement above the 64.28% majority-class baseline was approximately 14.22 percentage points in accuracy. More importantly, the 71.28% macro F1-score demonstrates that the model achieved a reasonable balance across all three sentiment classes rather than relying only on the negative majority. The selected configuration was therefore saved for production inference in the local application.



```

Model training - VS Code terminal

(.venv) % python -m src.train_model
MODEL COMPARISON
TF-IDF + Linear SVM      accuracy 0.7850   macro F1 0.7128
Bag of Words + Linear SVM accuracy 0.7715   macro F1 0.7047
Bag of Words + Multinomial NB accuracy 0.7761   macro F1 0.6909
TF-IDF + Multinomial NB  accuracy 0.7381   macro F1 0.5679
BEST MODEL: TF-IDF + Linear SVM
Saved: sentiment_model.pkl, vectorizer.pkl, model_metadata.json

```

Figure 5. Terminal evidence from the completed model-comparison run.

5.3 Functional Verification

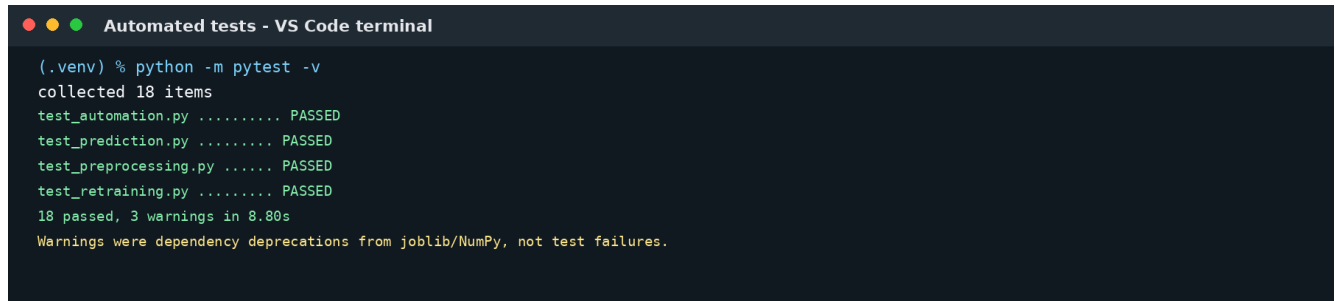
Verification area	Evidence	Outcome
Environment and imports	All required libraries imported successfully.	Passed
Preprocessing examples	Mentions/URLs removed; negation preserved; lemmatisation applied.	Passed
Data preparation	14,069 usable cleaned records generated.	Passed
Model training	Four model-feature combinations trained and	Passed

Verification area	Evidence	Outcome
	compared.	
Artifact persistence	Model, vectorizer and metadata saved.	Passed
GUI	Manual and batch workflows responded correctly.	Passed
Automation	New file detected, processed, saved and archived.	Passed
Automated tests	18 tests passed.	Passed

Table 8. Functional verification summary.

5.4 Automated Testing

The final pytest run collected 18 tests covering text preprocessing, prediction, file-column detection, alert thresholds, retraining-column detection and invalid-label handling. All tests passed. Three warnings were produced when joblib loaded the persisted NumPy arrays; these were dependency deprecation warnings and did not represent failed tests or incorrect predictions.



```

Automated tests - VS Code terminal
(.venv) % python -m pytest -v
collected 18 items
test_automation.py ..... PASSED
test_prediction.py ..... PASSED
test_preprocessing.py ..... PASSED
test_retraining.py ..... PASSED
18 passed, 3 warnings in 8.80s
Warnings were dependency deprecations from joblib/NumPy, not test failures.

```

Figure 9. Final automated test-suite evidence: 18 tests passed.

6. SYSTEM IMPLEMENTATION AND SCREENSHOTS

6.1 Desktop Dashboard

The dashboard combines manual prediction and batch reporting in one window. The upper input area accepts a review and provides Analyse Review, Upload CSV or JSON, Export Results and Clear Dashboard controls. Summary cards show total, positive, neutral and negative counts. The lower area contains a scrollable table and a live bar chart. Figure 6 is a direct macOS screenshot captured after customer_reviews.csv was uploaded; the application processed 14,408 usable reviews and displayed their aggregate sentiment distribution.

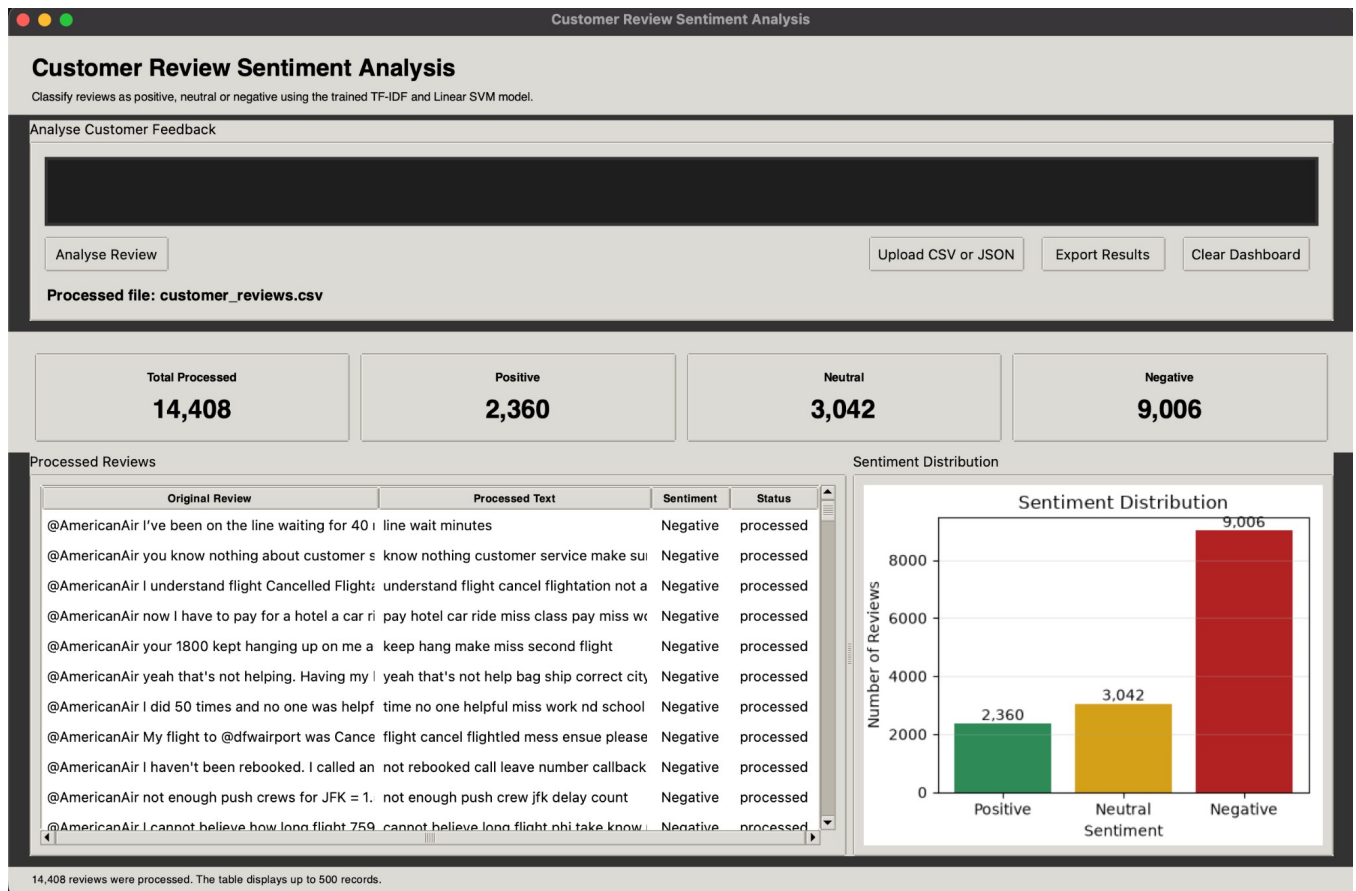


Figure 6. Actual macOS Tkinter dashboard after batch processing customer_reviews.csv, showing 14,408 processed reviews, sentiment summary cards, the review table and the distribution chart.

6.2 Automated File Processing

The folder monitor was started in continuous mode. After new_feedback.csv was placed in the incoming folder, the system detected the feedback column, classified all three records, correctly suppressed the negative alert because fewer than five reviews were present, saved the processed CSV and moved the source to the archive. The user then stopped the monitor with Control+C.

```
Automation monitor - VS Code terminal
Processing incoming file: new_feedback.csv
Detected review column: feedback
No alert generated. Only 3 processed reviews were available; at least 5 are required.
Processed reviews: 3
Skipped reviews: 0
Alert triggered: False
Results saved to: data/processed/automated/new_feedback_processed_20260731_124057.csv
Original archived: data/archive/new_feedback_20260731_124057.csv
^C Automation stopped by the user.
```

Figure 7. Actual automation-run output based on the completed terminal session.

6.3 Project Outputs

The final project generated the following core deliverables:

- Cleaned dataset: data/processed/customer_reviews_cleaned.csv.
- Best model: models/sentiment_model.pkl.
- Fitted vectorizer: models/vectorizer.pkl.
- Model metadata: models/model_metadata.json.
- Model-comparison report and four classification reports.
- Four confusion-matrix figures and two exploratory-analysis figures.
- Tkinter GUI launched through main.py.
- Automated incoming-file processor and archive workflow.
- Negative-review alert log.
- Controlled retraining reports and model backups.
- Automated test suite with 18 passing tests.
- Project README with setup, execution and limitation guidance.

7. LIMITATIONS AND FUTURE IMPROVEMENTS

7.1 Limitations

- Domain specificity: the model was trained on airline-related feedback and may perform less reliably on healthcare, retail, banking or other domains without domain-specific retraining.
- Class imbalance: negative feedback formed 64.28% of the cleaned dataset, which can influence the model even though class weighting and macro metrics were used.
- Language coverage: only English reviews are supported by the current preprocessing resources and training data.
- Nuance: sarcasm, humour, irony, mixed sentiment and context-dependent expressions remain difficult for a traditional linear classifier.
- Single-label output: each review receives one overall label even when it contains both praise and criticism.
- No calibrated confidence score: LinearSVC does not directly return probabilities without an additional calibration model.
- Local-only deployment: the Tkinter interface and folder monitor are suitable for demonstration but not a multi-user internet service.
- Local alerts: warnings are displayed or logged locally; email, SMS and collaboration-platform delivery were not connected.
- No production database: files are used for storage and archiving, which limits concurrency, auditing and long-term querying.
- Interface portability: the screenshot reflects the macOS Tkinter theme and display scaling; the appearance may vary on Windows, Linux or screens with different resolutions.

7.2 Future Improvements

- Collect and label feedback from the intended business domain, then evaluate performance on a domain-specific holdout set.
- Add probability calibration to provide prediction confidence and route uncertain cases for human review.
- Extend the classifier to multilingual feedback and test language-specific preprocessing.
- Implement aspect-based sentiment analysis to identify whether sentiment concerns price, staff, delays, product quality or another feature.
- Add topic modelling and trend analysis so management can see recurring complaints and changes over time.
- Replace the desktop-only interface with a web dashboard and REST API.
- Use PostgreSQL or another database for durable review history, audit trails and reporting.
- Use a message queue and background workers for higher-volume real-time processing.
- Integrate email, SMS, Slack or Microsoft Teams notifications.
- Package the system with Docker and deploy it to a managed cloud environment.
- Compare the current traditional models with transformer-based models such as BERT while considering computational cost and explainability.
- Add scheduled retraining, model-version governance, drift detection and rollback procedures.

8. CONCLUSION AND RECOMMENDATIONS

8.1 Conclusion

The project successfully produced an end-to-end automated sentiment analysis system that met the central requirements of the Learning Saint brief. A three-class customer-feedback dataset was sourced and prepared; a reusable preprocessing pipeline was developed; Bag of Words and TF-IDF were compared across Multinomial Naive Bayes and Linear SVM; the best model was persisted; and a user interface, file automation, alerts, retraining and testing modules were integrated.

TF-IDF with Linear SVM was the strongest configuration, achieving 78.50% accuracy and a 71.28% macro F1-score on 2,814 unseen reviews. This performance was materially stronger than the majority-class baseline and more balanced across the three sentiment classes than the alternatives. The system also demonstrated operational completeness: it responded correctly through the GUI, processed incoming files automatically, created reusable artifacts and passed all 18 automated tests.

The resulting application is suitable as a robust internship demonstration and a foundation for a business-facing sentiment analysis service. Its limitations are primarily those expected of a local, domain-specific, traditional machine-learning prototype. The next development stage should focus on domain-specific validation, calibrated confidence, persistent storage, external notifications and production deployment architecture.

8.2 Recommendations

9. Retain the direct dashboard screenshot, terminal evidence and automated-test results in the final submission to demonstrate the complete end-to-end workflow.
10. Include the complete source code, requirements.txt, README, saved model files and selected sample outputs in the submission folder.
11. Keep the current test set protected and avoid retraining on it.
12. Use macro F1-score and class-level recall, not accuracy alone, when future models are assessed.
13. Collect newly labelled reviews from the target industry before presenting the model as production-ready.
14. Record the video demonstration using a short end-to-end workflow: manual prediction, file upload, chart update, automation and tests.

REFERENCES

1. Learning Saint. (2026). Client Requirement for Automated Sentiment Analysis System and Sentiment Analysis GUI with Tkinter - Comprehensive Guide. Unpublished internship project brief.
2. CrowdFlower. (n.d.). Twitter US Airline Sentiment. Kaggle.
<https://www.kaggle.com/datasets/crowdflower/twitter-airline-sentiment>
3. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
<https://jmlr.org/papers/v12/pedregosa11a.html>
4. Scikit-learn developers. (2026). TfidfVectorizer documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
5. Scikit-learn developers. (2026). MultinomialNB documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html
6. Scikit-learn developers. (2026). LinearSVC documentation.
<https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>
7. NLTK Project. (2023). WordNetLemmatizer documentation.
<https://www.nltk.org/api/nltk.stem.WordNetLemmatizer.html>
8. Python Software Foundation. (2026). tkinter - Python interface to Tcl/Tk.
<https://docs.python.org/3/library/tkinter.html>
9. Scikit-learn developers. (2026). classification_report documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html
10. Wang, S., & Manning, C. D. (2012). Baselines and Bigrams: Simple, Good Sentiment and Topic Classification. Proceedings of ACL 2012. <https://aclanthology.org/P12-2018/>

APPENDICES

Appendix A: Key Commands

```
# Activate the environment
source .venv/bin/activate

# Run the desktop dashboard
python main.py

# Prepare and clean the dataset
python -m src.prepare_dataset
python -m src.clean_dataset
python -m src.explore_data

# Train and compare models
python -m src.train_model

# Process available incoming files once
python -m src.automation --once

# Monitor continuously
python -m src.automation

# Retrain with new labelled feedback
python -m src.retrain_model --input data/retraining/new_labeled_reviews.csv

# Run tests
python -m pytest -v
```

Appendix B: Project Structure

```
Sentiment Analysis & Naive Bayes/
├── data/
│   ├── archive/
│   ├── incoming/
│   ├── processed/
│   ├── raw/
│   └── retraining/
├── models/
│   ├── backups/
│   ├── model_metadata.json
│   ├── sentiment_model.pkl
│   └── vectorizer.pkl
├── reports/
│   ├── figures/
│   ├── retraining/
│   └── model_comparison.csv
├── src/
│   ├── automation.py
│   ├── clean_dataset.py
│   ├── explore_data.py
│   ├── gui.py
│   ├── notification.py
│   └── predict.py
```

```
| |— prepare_dataset.py  
| |— preprocessing.py  
| |— retrain_model.py  
| |— train_model.py  
|— tests/  
|— main.py  
|— README.md  
|— requirements.txt
```

Appendix C: Submission Checklist

- Complete source-code folder with documented modules.
- requirements.txt and README.md.
- customer_reviews_cleaned.csv or clear dataset download/preparation instructions.
- sentiment_model.pkl and vectorizer.pkl.
- Model comparison, classification reports and figures.
- Formal project report in PDF format.
- Five-to-ten-minute presentation video.
- Direct screenshot of the running Tkinter dashboard included as Figure 6.
- Final verification using `python -m pytest -q`.